# Network Edges Inference

This document provides a unified framework for performing Bayesian inference of social networks based on observational data, implementing the BISoN framework methodologies Hart et al. (2023). Real-world social network connections (edges) are often inferred from observational data where uniform sampling over all pairs is not always possible. This framework enables modeling edge weights with uncertainty by decomposing aggregated observations into the generative sampling processes.

Below, we detail the implementation of various edge weight models for different types of observational data using the Bayesian Inference (**BI**) package via Python, R, and Julia.

## General Principles

The fundamental principle behind the Network Edges Inference models is to capture uncertainty in edge weights by modeling the data collection process directly, rather than relying on point estimates (e.g., Simple Ratio Index). By decomposing the data into distinct observation periods, these models can inherently capture the uncertainty generated by sampling effort and allow for observation-level effects (such as variable visibility) to be controlled.

Three main types of observational data are supported: 1. **Binary data**: Presence/absence of social events in fixed sampling periods. 2. **Count data**: Number of social events per sampling period. 3. **Duration data**: Amount of time spent engaged in social events within a sampling period.

---

## Binary Data Model

### General Principles

The binary data model is used when the presence or absence of a social event for each dyad is recorded in each sampling period. The edge weight represents the probability of (or proportion of time) engaging in a social event in a fixed period of time.

### Considerations

> **Note**
>
> - **Priors**: The edge weight parameters normally represent logit-scale proportions and typically use standard Normal priors relative to prior structural beliefs (e.g., `Normal(0, 2)`).
> - **Data**: The model uses the number of sampling periods (`divisor`) where an event could occur and the number of observed events (`event`).
> - **Fixed Effects**: Additional variables accounting for fixed effects (e.g., location or age difference) can be added to the predictor iteratively.

### Example with simulated data

## Python

```
from BI import bi

m = bi(platform='cpu')
m.data_on_model = {'data': data_list_jax} # Must contain 'prior_edge_mu', 'num_edges', 'dyad

def bi_model_binary(data):
    # Edge weights prior
    edge_weight = m.dist.normal(data['prior_edge_mu'], data['prior_edge_sigma'],
                                shape=(data['num_edges'],), name="edge_weight")

    # 0-based indexing for JAX
    dyad_ids_0 = data['dyad_ids'] - 1

    # Build predictor
    predictor = edge_weight[dyad_ids_0]

    # Add fixed effects if applicable
    if data['num_fixed'] > 0:
        beta_fixed = m.dist.normal(data['prior_fixed_mu'], data['prior_fixed_sigma'],
                                   shape=(data['num_fixed'],), name="beta_fixed")
        predictor = predictor + m.jnp.dot(data['design_fixed'], beta_fixed)

    # Convert from logit scale to probability
    p = m.jax.scipy.special.expit(predictor)
```

```
    # Binomial Likelihood
    m.dist.binomial(data['divisor'], p, obs=data['event'], name="event")

m.fit(bi_model_binary)
m.summary()
```

**R**

```
library(BayesianInference)
m <- importBI(platform = "cpu")
jnp <- reticulate::import("jax.numpy")
jax_scipy <- reticulate::import("jax.scipy.special")

m$data_on_model <- list(data = data_list_jax)

bi_model_binary <- function(data) {
  # Edge weights prior
  edge_weight <- m$dist$normal(data$prior_edge_mu, data$prior_edge_sigma,
                               shape=tuple(data$num_edges), name="edge_weight")

  # 0-based indexing for JAX
  dyad_ids_0 <- data$dyad_ids - 1L

  # Build predictor
  predictor <- edge_weight[dyad_ids_0]

  if (data$num_fixed > 0) {
    beta_fixed <- m$dist$normal(data$prior_fixed_mu, data$prior_fixed_sigma,
                                shape=tuple(data$num_fixed), name="beta_fixed")
    predictor <- predictor + jnp$dot(data$design_fixed, beta_fixed)
  }

  p <- jax_scipy$expit(predictor)

  m$dist$binomial(data$divisor, p, obs=data$event, name="event")
}

m$fit(bi_model_binary)
m$summary()
```

### Julia

```
using BayesianInference

m = importBI(platform="cpu")
m.data_on_model = Dict("data" => data_list_jax)

@BI function bi_model_binary(data)
    edge_weight = m.dist.normal(data["prior_edge_mu"], data["prior_edge_sigma"],
                                shape=(data["num_edges"],), name="edge_weight")

    # 0-based indexing for JAX inside the environment
    dyad_ids_0 = data["dyad_ids"] .- 1

    predictor = edge_weight[dyad_ids_0]

    if data["num_fixed"] > 0
        beta_fixed = m.dist.normal(data["prior_fixed_mu"], data["prior_fixed_sigma"],
                                   shape=(data["num_fixed"],), name="beta_fixed")
        predictor = predictor .+ jnp.dot(data["design_fixed"], beta_fixed)
    end

    p = jax.scipy.special.expit(predictor)

    m.dist.binomial(data["divisor"], p, obs=data["event"], name="event")
end

m.fit(bi_model_binary)
m.summary()
```

### Mathematical Details

For binary data, the mathematical process applies a Binomial likelihood assuming multiple Bernoulli trials:

$$X_{ij}^{(n)} \sim \text{Binomial}(D_{ij}^{(n)}, p_{ij}^{(n)})$$

Where $X_{ij}^{(n)}$ is the presence/absence in the $n$-th observation, $D_{ij}^{(n)}$ is the condition (often 1 or representing multiple grouped trials), and $p_{ij}^{(n)}$ is the dyad interaction probability. The model logit-links the predictor effects with the primary edge weight:

$$\text{logit}(p_{ij}^{(n)}) = \omega_{ij} + \ldots$$

Where $\omega_{ij}$ represents the foundational edge weight parameter for dyad $i, j$.

### Notes

> **Note**
>
> The binary binomial implementation directly provides a framework mirroring the Simple Ratio Index (SRI) while preserving variance from low sample depth.

---

## Count Data Model

### General Principles

The count data model interprets instances where the exact *number of events* for each dyad is recorded in each sampling period. Under this formulation, the model outputs a rate of occurrence of social events per unit of time instead of a bounded probability.

### Considerations

> **Note**
>
> - **Priors**: Counts use a standard Poisson process; therefore, edge predictors dictate the log-rate ($\lambda$) and must be carefully scaled relative to time. `Normal` priors on the log-rate parameter effectively capture the magnitudes.
> - **Link Function**: Uses the exponential function applied to the linear predictor configuration.

### Example with simulated data

## Python

```
from BI import bi

m = bi(platform='cpu')
m.data_on_model = {'data': data_list_jax}

def bi_model_count(data):
    # Edge weights prior (log-rate)
    edge_weight = m.dist.normal(data['prior_edge_mu'], data['prior_edge_sigma'],
```

```
                                shape=(data['num_edges'],), name="edge_weight")

    dyad_ids_0 = data['dyad_ids'] - 1
    predictor = edge_weight[dyad_ids_0]

    if data['num_fixed'] > 0:
        beta_fixed = m.dist.normal(data['prior_fixed_mu'], data['prior_fixed_sigma'],
                                   shape=(data['num_fixed'],), name="beta_fixed")
        predictor = predictor + m.jnp.dot(data['design_fixed'], beta_fixed)

    # Scale via event lengths/time blocks
    rate = m.jnp.exp(predictor) * data['divisor']

    # Poisson Likelihood
    m.dist.poisson(rate, obs=data['event'], name="event")

m.fit(bi_model_count)
m.summary()
```

**R**

```
library(BayesianInference)
m <- importBI(platform = "cpu")
jnp <- reticulate::import("jax.numpy")

m$data_on_model <- list(data = data_list_jax)

bi_model_count <- function(data) {
  edge_weight <- m$dist$normal(data$prior_edge_mu, data$prior_edge_sigma,
                               shape=tuple(data$num_edges), name="edge_weight")

  dyad_ids_0 <- data$dyad_ids - 1L
  predictor <- edge_weight[dyad_ids_0]

  if (data$num_fixed > 0) {
    beta_fixed <- m$dist$normal(data$prior_fixed_mu, data$prior_fixed_sigma,
                                shape=tuple(data$num_fixed), name="beta_fixed")
    predictor <- predictor + jnp$dot(data$design_fixed, beta_fixed)
  }

  rate <- jnp$exp(predictor) * data$divisor
```

```
  m$dist$poisson(rate, obs=data$event, name="event")
}

m$fit(bi_model_count)
m$summary()
```

### Julia

```
using BayesianInference

m = importBI(platform="cpu")
m.data_on_model = Dict("data" => data_list_jax)

@BI function bi_model_count(data)
    edge_weight = m.dist.normal(data["prior_edge_mu"], data["prior_edge_sigma"],
                                shape=(data["num_edges"],), name="edge_weight")

    dyad_ids_0 = data["dyad_ids"] .- 1
    predictor = edge_weight[dyad_ids_0]

    if data["num_fixed"] > 0
        beta_fixed = m.dist.normal(data["prior_fixed_mu"], data["prior_fixed_sigma"],
                                   shape=(data["num_fixed"],), name="beta_fixed")
        predictor = predictor .+ jnp.dot(data["design_fixed"], beta_fixed)
    end

    rate = jnp.exp(predictor) .* data["divisor"]

    m.dist.poisson(rate, obs=data["event"], name="event")
end

m.fit(bi_model_count)
m.summary()
```

#### Mathematical Details

For count data, we use the sum of Poisson-distributed random variables to define interactions scaling with observation duration.

$$X_{ij}^{(n)} \sim \text{Poisson}(\lambda_{ij}^{(n)} D_{ij}^{(n)})$$

Where the overall rate modifier is described logarithmically:

$$\log(\lambda_{ij}^{(n)}) = \omega_{ij} + ...$$

Where $\omega_{ij}$ evaluates to the maximum likelihood estimation counterpart of straightforward event rate counts across cumulative time blocks.

### Notes

> **Note**
>
> The Poisson distribution inherently manages the aggregation mechanics allowing the edge weight to serve dynamically scaled estimates independent from disparate observation lengths.

---

## Duration Data Model

### General Principles

The duration model simultaneously handles two data metrics: the *duration* of social events and the *frequency* with which they occur. It treats total engaged time mathematically using a composition approach tracking exponential duration periods bounded across a baseline Poisson event initiation rate.

### Considerations

> **Note**
>
> - **Priors**: Requires *two* primary priors. The standard edge-weight proportion models the relative event time ($\omega$), while an overarching generalized `rate` component drives the base observation scale.
> - **Complexity**: Fits two separate likelihood distributions (Poisson and Exponential) interacting across shared predictors to map both count frequencies and continuous lengths simultaneously.

### Example with simulated data

### Python

```
from BI import bi

m = bi(platform='cpu')
m.data_on_model = {'data': data_list_jax}

def bi_model_duration(data):
    # Edge weights (proportion scale mapping)
    edge_weight = m.dist.normal(data['prior_edge_mu'], data['prior_edge_sigma'],
                                shape=(data['num_edges'],), name="edge_weight")

    # Additional absolute rate parameter strictly positive (Half-Normal equivalent constraint
    rate_param = m.dist.normal(0.0, data['prior_rate_sigma'],
                               shape=(data['num_edges'],), name="rate")
    rate_positive = m.jnp.abs(rate_param)

    dyad_ids_0 = data['dyad_ids'] - 1
    predictor = edge_weight[dyad_ids_0]

    if data['num_fixed'] > 0:
        beta_fixed = m.dist.normal(data['prior_fixed_mu'], data['prior_fixed_sigma'],
                                   shape=(data['num_fixed'],), name="beta_fixed")
        predictor = predictor + m.jnp.dot(data['design_fixed'], beta_fixed)

    # Convert to expected total scale fraction
    p = m.jax.scipy.special.expit(predictor)

    # Build linked parameters
    lambda_exp = rate_positive[dyad_ids_0] / p
    lambda_pois = rate_positive * data['divisor']

    # Joint likelihood inference
    m.dist.exponential(lambda_exp, obs=data['event'], name="event")
    m.dist.poisson(lambda_pois, obs=data['event_count'], name="event_count")

m.fit(bi_model_duration)
m.summary()
```

### R

```r
library(BayesianInference)
m <- importBI(platform = "cpu")
jnp <- reticulate::import("jax.numpy")
jax_scipy <- reticulate::import("jax.scipy.special")

m$data_on_model <- list(data = data_list_jax)

bi_model_duration <- function(data) {
  edge_weight <- m$dist$normal(data$prior_edge_mu, data$prior_edge_sigma,
                               shape=tuple(data$num_edges), name="edge_weight")

  rate_param <- m$dist$normal(0.0, data$prior_rate_sigma,
                              shape=tuple(data$num_edges), name="rate")
  rate_positive <- jnp$abs(rate_param)

  dyad_ids_0 <- data$dyad_ids - 1L
  predictor <- edge_weight[dyad_ids_0]

  if (data$num_fixed > 0) {
    beta_fixed <- m$dist$normal(data$prior_fixed_mu, data$prior_fixed_sigma,
                                shape=tuple(data$num_fixed), name="beta_fixed")
    predictor <- predictor + jnp$dot(data$design_fixed, beta_fixed)
  }

  p <- jax_scipy$expit(predictor)

  lambda_exp <- rate_positive[dyad_ids_0] / p
  lambda_pois <- rate_positive * data$divisor

  m$dist$exponential(lambda_exp, obs=data$event, name="event")
  m$dist$poisson(lambda_pois, obs=data$event_count, name="event_count")
}

m$fit(bi_model_duration)
m$summary()
```

### Julia

```
using BayesianInference

m = importBI(platform="cpu")
m.data_on_model = Dict("data" => data_list_jax)

@BI function bi_model_duration(data)
    edge_weight = m.dist.normal(data["prior_edge_mu"], data["prior_edge_sigma"],
                                shape=(data["num_edges"],), name="edge_weight")

    rate_param = m.dist.normal(0.0, data["prior_rate_sigma"],
                               shape=(data["num_edges"],), name="rate")
    rate_positive = jnp.abs(rate_param)

    dyad_ids_0 = data["dyad_ids"] .- 1
    predictor = edge_weight[dyad_ids_0]

    if data["num_fixed"] > 0
        beta_fixed = m.dist.normal(data["prior_fixed_mu"], data["prior_fixed_sigma"],
                                   shape=(data["num_fixed"],), name="beta_fixed")
        predictor = predictor .+ jnp.dot(data["design_fixed"], beta_fixed)
    end

    p = jax.scipy.special.expit(predictor)

    lambda_exp = rate_positive[dyad_ids_0] ./ p
    lambda_pois = rate_positive .* data["divisor"]

    m.dist.exponential(lambda_exp, obs=data["event"], name="event")
    m.dist.poisson(lambda_pois, obs=data["event_count"], name="event_count")
end

m.fit(bi_model_duration)
m.summary()
```

### Mathematical Details

The mean behavior time $\mu_{ij}$ can be recovered from the estimated proportion scale models representing expected duration lengths per sequence generated by overarching absolute rates $\lambda_{ij}$.

$$X_{ij}^{(n)} \sim \text{Exponential}(\lambda_{ij}/t_{ij}^{(n)})$$

$$K_{ij} \sim \text{Poisson}(\lambda_{ij} D_{ij})$$

The proportion is generated matching logistic mapping inputs:

$$\text{logit}(t_{ij}^{(n)}) = \omega_{ij} + ...$$

And recovered as expected proportion values.

### Notes

> **Note**
>
> Because edge estimates define probability scales while absolute intensity generates sequence frequency, parameter isolation allows complex uncoupled insights such as distinct testing comparing the absolute magnitude of interactions versus simply calculating engaged durations.

Hart, Jordan, Michael Nash Weiss, Daniel Franks, and Lauren Brent. 2023. "BISoN: A Bayesian Framework for Inference of Social Networks." *Methods in Ecology and Evolution* 14: 2411–20. https://doi.org/https://doi.org/10.1111/2041-210X.14171.